SOFTWARE REQUIREMENTS SPECIFICATION

for

PharmaFlowAI

Pharmacy Management System

Version 1.0

Prepared by:

Name	Roll number
Sahil	1024240063
Takshit	1024240056
Sidak	1024240043
Ridhmann	1024240059

Submitted to: [Supervisor name and designation]

AIML
Thapar Institute of Engineering and Technology
UCS503 — Software Engineering [Submission date]

Implementation baseline reviewed: 12 October 2026.
Supervisor and submission-date fields remain editable pending confirmation.

Document Control

Catalogue, reports and bilingual assistance; operational modules remain session prototypes.

Table 0.1: Document control information

Field	Value
Document	PharmaFlowAI Software Requirements Specification
Version / baseline	1.0 / 12 October 2026
Purpose	Academic assessment, maintenance and verifiable scope definition
Implementation status	Implemented, Prototype, Proposed and To verify are kept distinct; test results are bounded evidence.
Technology baseline	Python 3.13.9, Django 6.1; SQLite default, PostgreSQL configurable; Django templates and JavaScript; browser-dependent speech.
Academic details	AIML; Thapar Institute of Engineering and Technology; UCS503 — Software Engineering
Pending metadata	[Supervisor name and designation]; [Submission date]

Table 0.2: Document revision record

Version	Date	Change
1.0	12 October 2026	Initial implementation-grounded SRS using the approved reference structure. Includes catalogue reports and the signed-in bilingual voice assistant.

Editorial revision note (version unchanged). Diagram styling, explicit requirement priorities/statuses, permissions, traceability and acceptance planning are expanded. No implementation change or new test execution is asserted. The implementation baseline remains 12 October 2026.

Contents

Document Control **2**

1 Introduction **6**

1.1 Purpose . . . 6
1.2 Intended Audience and Reading Suggestions . . . 6
1.3 Project Scope . . . 6
1.4 Definitions, Acronyms and Abbreviations . . . 7
1.5 References . . . 8

2 Overall Description **10**

2.1 Product Perspective . . . 10
2.2 User Classes and Characteristics . . . 10
2.3 Product Functions . . . 12
2.4 Operating Environment . . . 17
2.5 Design . . . 18

3 System Features **23**

3.1 Description and Priority . . . 23
3.2 Functional Requirements . . . 23

4 Other Nonfunctional Requirements **31**

4.1 Performance Requirements . . . 31
4.2 Security Requirements . . . 31
4.3 Software Quality Attributes . . . 33
4.4 Business Rules . . . 34

5 Other Requirements **35**

5.1 Assumptions and Dependencies . . . 35
5.2 Maintenance and Proposed Enhancements . . . 35
5.3 Verification and Acceptance Mapping . . . 36
5.4 Reference Alignment and Document Control . . . 38
5.5 Source References . . . 39
5.6 Terminology . . . 39

List of Figures

1.1 Current workflow. Authenticated catalogue functions use persistent medicine data; operational demo records remain session-scoped. 7

2.1 User classification. Administration is a permission-controlled capability, not a separate pharmacy business actor. 11

2.2 DFD Level 0. Customers use an authenticated counter session. Supplier details are entered by the Owner/Pharmacist; no supplier portal exists. Audio is processed through browser speech services, not uploaded to Django. 13

2.3 DFD Level 1. Repeated actors and D2 symbols denote the same participant or store. D1 contains framework sessions and the prototype payload, not an operational ledger. Processes 2.0–5.0 require an authenticated session; speech conversion surrounds process 4.0 in the browser. 14

2.4 Class/ER view of the real domain and framework session store. Session attributes are shown conceptually because the source SRS does not enumerate its physical schema. 16

2.5 Layered component architecture. All application modules share the Django monolith. WSGI/ASGI hooks and PostgreSQL configuration do not establish production deployment. . 18

2.6 Catalogue creation/edit activity. Invalid submissions retain the form. Separate POST status actions deactivate or reactivate records; application forms do not delete medicines. 20

2.7 Voice-assistant sequence with activation bars. Recognition and speech synthesis are browser-dependent; Django receives reviewed text and language only. 21

2.8 Medicine state diagram. POST status actions update the flag and timestamp without deleting the record. 22

2.9 Assistant request states. Typed input bypasses Listening; recognition never sends automatically. 22

3.1 Use cases and access boundary. The dashed prototype association denotes business participation, not direct system authorization. 28

List of Tables

0.1 Document control information . . . 2
0.2 Document revision record . . . 2
1.1 Diagram colour legend . . . 7
1.2 Definitions and abbreviations . . . 7
2.1 User classes and actual authorization boundaries . . . 10
2.2 Users and permissions in the current baseline . . . 11
2.3 Module responsibilities and persistence . . . 12
2.4 Medicine entity data dictionary . . . 15
2.5 Verified technology baseline and conditional dependencies . . . 17
2.6 Principal HTTP interfaces . . . 19
3.1 Feature priorities . . . 23
3.2 Authentication requirements and acceptance criteria . . . 23
3.3 Catalogue and dashboard requirements . . . 24
3.4 Catalogue reporting requirements . . . 25
3.5 Prototype module requirements . . . 26
3.6 Voice-assistant requirements . . . 26
3.7 Representative use-case specifications . . . 29
4.1 Performance and capacity requirements . . . 31
4.2 Security requirements and limitations . . . 32
4.3 Quality attributes and verifiable expectations . . . 33
5.1 Future work, explicitly outside the implemented baseline . . . 35
5.2 Requirement-to-verification mapping . . . 36
5.3 Traceability: authentication, catalogue, reports and prototypes . . . 37
5.4 Traceability: chatbot, voice and outstanding nonfunctional checks . . . 38

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the current scope, externally observable behaviour, data requirements and quality expectations of PharmaFlowAI. The application supports a small pharmacy through authenticated access to a medicine master catalogue, catalogue summaries, printable reports and a bilingual voice/text assistant. Additional operational modules demonstrate possible workflows using session-scoped sample data.

This document is grounded in the project working directory reviewed on 12 October 2026, including current uncommitted files. Source code, templates, configuration and automated tests determine the implementation baseline. Comments describing future inventory or billing behaviour are not treated as evidence that those capabilities exist.

The SRS provides a common reference for academic assessment, maintenance and further development. Requirements use *shall* for verifiable expected behaviour and distinguish the following states throughout:

- **Implemented**: behaviour supported by the reviewed implementation; this label does not imply production certification or exhaustive testing.
- **Prototype**: executable demonstration behaviour based on session-specific sample records, not a completed operational subsystem.
- **Proposed**: a future requirement or measurable target not established as implemented or achieved.
- **To verify**: a target, environment-dependent behaviour or acceptance check for which measured evidence is not established. This does not by itself imply an unimplemented feature.

Requirement identifiers such as FR-AUTH-01 and NFR-SEC-01 are stable references for verification. Their priority indicates importance within the stated scope, not completion status.

1.2 Intended Audience and Reading Suggestions

Academic evaluators and project supervisors should read Sections 1–3 to understand scope, design and functional coverage. The Owner/Pharmacist should consult Sections 2.2, 2.3 and 4.4 for supported activities and business limitations. Developers should read the interfaces, data model and source references in Sections 2 and 1.5. Testers should use the acceptance criteria in Section 3.2 and the verification mapping in Section 5.3.

The term *customer assistant* describes the audience of the assistance feature. It does not establish a separate customer account type, public portal or ordering service. The feature is named *Voice assistant* in the user interface and is available only inside the signed-in application.

1.3 Project Scope

The implemented scope comprises login with email or username; configurable session persistence; an authenticated dashboard; medicine creation, inspection, editing and status changes; catalogue searching, filtering and pagination; a complete catalogue report with CSV export and browser printing; and English/Hindi catalogue assistance with optional browser speech input and output.

The inventory, billing, supplier and customer screens support sample-record addition, editing, deletion, client-side search and export. Their data is serialized inside the user's Django session. Analytics displays predefined demonstration metrics. These interfaces do not provide a persistent, shared business ledger, and their displayed figures are not measurements of actual pharmacy performance.

The scope excludes completed batch inventory, verified stock availability, price management, expiry calculations, sales transactions, payment processing, tax calculation, clinical decision support and generative AI. Independent supplier/customer portals and email-based password recovery are also not implemented.

PostgreSQL configuration is available, but a deployed PostgreSQL environment is not evidenced by the project.

Table 1.1: Diagram colour legend

Colour	Meaning
Blue	Presentation and user interface
Green	Application logic and processes
Yellow	Data stores and database; key-rule notes
Orange	Session-scoped prototype modules
Red	External boundaries, errors and limitations
Purple	Actors and users
Grey	Explanatory notes and diagram boundaries

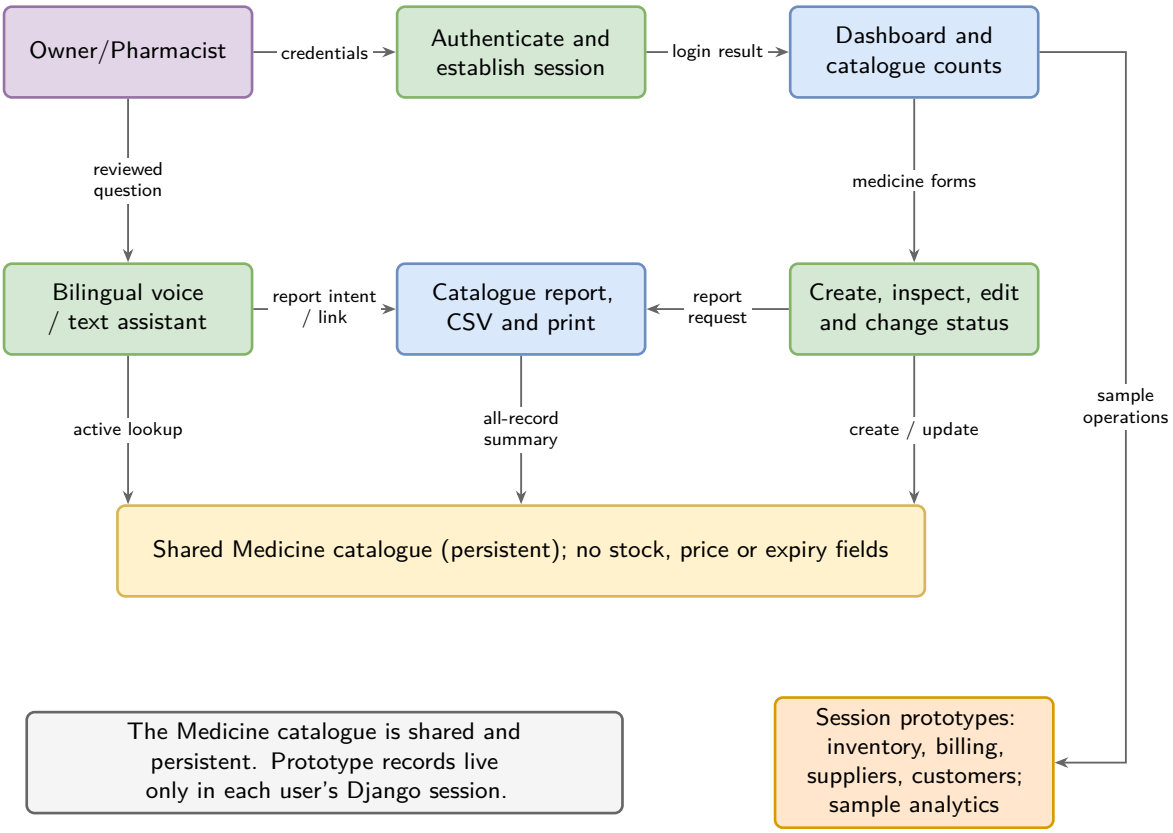


Figure 1.1: Current workflow. Authenticated catalogue functions use persistent medicine data; operational demo records remain session-scoped.

1.4 Definitions, Acronyms and Abbreviations

Table 1.2: Definitions and abbreviations

Term	Definition
SRS	Software Requirements Specification.
CSV	Comma-separated values, used for the catalogue export.

Term	Definition
DFD	Data Flow Diagram: logical processes, data stores, external participants and exchanged information.
Catalogue	Master medicine descriptions and flags; it does not represent physical stock.
Session	Authenticated browser interaction identified by a session cookie, with server-side data in Django's configured session store.
Prototype record	Sample operational data serialized in a session, without its own shared business-domain table.
ORM	Object-relational mapper used by Django to access the configured relational database.
CSRF	Cross-Site Request Forgery; Django's CSRF token/middleware checks protect unsafe requests.
Voice assistant	Bounded, rule-based catalogue assistance with optional browser speech recognition and synthesis.
Acceptance criterion	A specific observation used to determine whether a stated requirement is satisfied under defined conditions.
RPO / RTO	Recovery point objective / recovery time objective; both remain to be agreed.
LLM	Large language model; no external LLM is used by this baseline.
API / HTTP	Application programming interface / Hypertext Transfer Protocol.

1.5 References

R01 IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*. Historical structural reference, not a certification claim. https://ieeexplore.ieee.org/document/720574.

R02 Django Software Foundation, *Django documentation*. Consult documentation matching the recorded project version; this link does not change the baseline. https://docs.djangoproject.com/.

R03 Python Software Foundation, *Python 3.13 documentation*. https://docs.python.org/3.13/.

R04 Supplied *SAMS SRS v2.0*, used only for diagram appearance and structural ideas in this revision. None of its domain content is adopted.

Source identifiers below refer to the project working-tree files reviewed for this document. They identify evidence, not third-party product guarantees. File paths are relative to the project root; the supplied template is an external reference. The reference's project-specific statements are not authoritative for PharmaFlowAI.

S01 **Reference SRS.** Supplied file `ryktphvctsxg.pdf`, "Software Requirements Specification for IICT Website", version 1.0, dated 24 August 2019, 11 pages. Used for chapter hierarchy and presentation only.

S02 **Configuration and dependency baseline.** `requirements.txt`; `config/settings.py`; `config/urls.py`; `config/wsgi.py`; `config/asgi.py`; `manage.py`; `.env.example`; `.claude/launch.json`.

S03 **Accounts.** `apps/accounts/models.py`; `apps/accounts/backends.py`; `apps/accounts/forms.py`; `apps/accounts/views.py`; `apps/accounts/urls.py`; `apps/accounts/admin.py`; initial account migration.

S04 **Medicine domain.** `apps/medicines/models.py`; `apps/medicines/forms.py`; `apps/medicines/views.py`; `apps/medicines/urls.py`; `apps/medicines/admin.py`; initial medicine migration.

S05 **Dashboard and prototypes.** `apps/core/views.py`; `apps/core/urls.py`; `templates/core/dashboard.html`; `templates/core/module.html`; `static/js/app.js`.

S06 **Catalogue report.** `apps/core/catalogue_report.py`; report view in `apps/core/views.py`; `templates/core/catalogue_report.html`; `static/css/catalogue_report.css`; `static/js/catalogue_report.js`.

S07 **Assistant service and interface.** `apps/core/assistant.py`; chat view in `apps/core/views.py`; `static/js/assistant.js`; `templates/partials/_assistant.html`; `docs/customer-assistant.md`.

S08 **Shared presentation.** `templates/base.html`; `templates/base_app.html`; `templates/partials/_sidebar.html`; account and medicine templates; `static/css/tokens.css`; `static/css/base.css`;

`static/css/app.css`; `static/css/assistant.css`.

S09 **Django verification sources.** `apps/accounts/tests/test_login.py`; `apps/medicines/tests/test_models.py`; `apps/medicines/tests/test_views.py`; `apps/core/tests.py`.

S10 **Frontend verification source.** `tests/assistant.test.cjs`. Speech event tests use simulation, not live microphone capture.

2 Overall Description

2.1 Product Perspective

PharmaFlowAI is a server-rendered web application implemented as a Django monolith. The browser renders Django templates and static HTML, CSS and JavaScript assets. Normal application forms use HTTP requests and server redirects. The voice assistant adds an asynchronous JSON request to the authenticated `/chat/` endpoint.

Three project applications are installed: `accounts`, `medicines` and `core`. Django supplies authentication, administration, sessions, messages, static-file handling and an object-relational mapper (ORM). There is no separate React client, REST framework, mobile application, message broker or external language-model service in the reviewed repository.

The default data store is SQLite. Setting PostgreSQL environment variables switches the configured database backend. Django's default database session backend also stores session records; consequently, the demo modules are *session-scoped server data*, not relational inventory or billing tables and not merely browser local storage. Medicine records form one shared catalogue; the schema does not associate each medicine with a particular account or pharmacy tenant.

2.2 User Classes and Characteristics

The Owner/Pharmacist is one combined functional actor in this SRS and performs the pharmacy's authenticated operational activities. Supplier and Customer are additional business participants, represented by prototype contact records and assisted interactions; they do not have independent account types or portals in the current application. Django administration is a permission-controlled capability of an authorized account, not a separate business actor.

Table 2.1: User classes and actual authorization boundaries

User class	Typical activities	Current access distinction
Owner/Pharmacist	Maintain catalogue, inspect dashboard, use reports and assistant, and manage prototype supplier/customer records.	Ordinary views require authentication. Access to `/admin/` additionally depends on Django staff status and permissions.
Supplier	Provide supplier contact information for pharmacy record keeping.	Represented by session-scoped prototype records maintained by the Owner/Pharmacist; no supplier login or portal.
Customer	Receive assistance with catalogue questions by text or voice at an authenticated counter session.	Contact records are session-scoped prototypes. No customer login or public assistant route is implemented.
Unauthenticated visitor	Open sign-in and password-help pages.	Protected pages redirect to login; the assistant endpoint returns an authentication error.

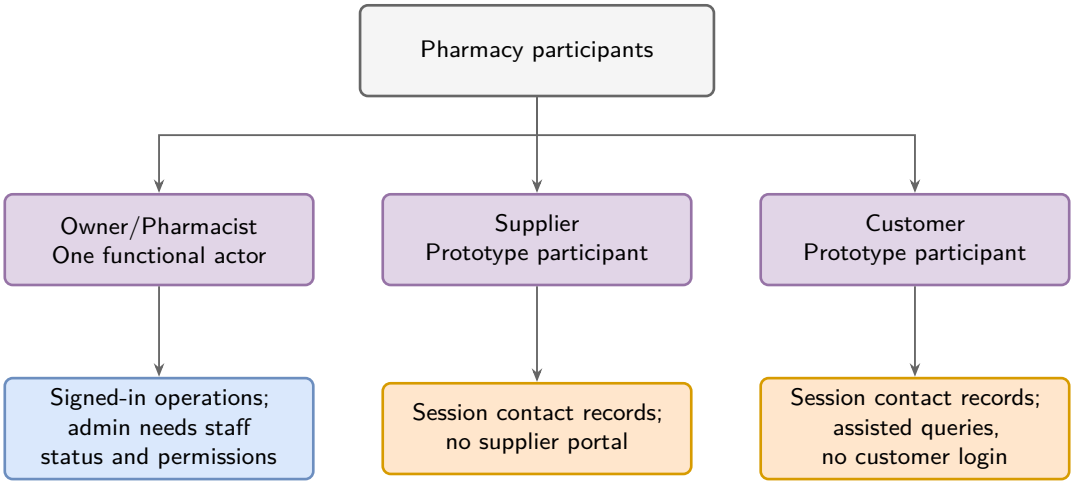


Figure 2.1: User classification. Administration is a permission-controlled capability, not a separate pharmacy business actor.

2.2.1 Users and Permissions

Table 2.2: Users and permissions in the current baseline

Resource / operation	Unauthenticated visitor	Owner/Pharmacist	Admin (staff / permissions)
Medicine application screens	No	C/R/U/A	C/R/U/A
Medicine deletion in application screens	No	No	No
Catalogue report / CSV	No	R	R
Voice/text assistant	No	Yes	Yes
Writable prototype modules	No	C/R/U/D	C/R/U/D
Analytics sample module	No	R	R
Django admin	No	No[1]	Yes[2]
Login with an active account	Yes	Yes	Yes

Matrix notes. C = create, R = read, U = update, D = delete, A = activate/deactivate. Prototype C/R/U/D changes only session data. Application medicine operations use C/R/U/A, not deletion. Anonymous page requests redirect to login; anonymous chat returns 401 unless CSRF rejects it first. Login availability does not imply public registration.

[1]The combined Owner/Pharmacist actor does not itself confer Django admin access. [2]An active staff account may enter Django admin, but individual model operations depend on its assigned permissions; a superuser has framework-level privileges. Admin medicine deletion may be available with the appropriate permission and is not prohibited by the application-screen rule. Supplier/Customer participants have no independent login or portal. The stored `role` label is not an enforced authorization policy; NFR-SEC-06 remains Proposed.

2.3 Product Functions

2.3.1 Implemented and prototype modules

Table 2.3: Module responsibilities and persistence

Module	Status	Responsibility and data source
Accounts	Implemented	Custom Django user, email/username authentication, login/logout and manual password-help guidance.
Medicines	Implemented	Database-backed catalogue with normalization, duplicate checks, filtering, details, editing and active/inactive status.
Dashboard	Implemented / mixed	Real medicine counts and recent additions alongside navigation and sample-module summaries.
Catalogue report	Implemented	All active and inactive medicines, category counts, prescription flags, CSV and print layout.
Voice assistant	Implemented	Deterministic catalogue queries and report intents; English/Hindi UI and replies; browser speech APIs.
Inventory	Prototype	Medicine label, quantity, reorder level, status and shelf fields stored as session sample records.
Billing	Prototype	Invoice, customer, item description, amount and status strings; no transaction calculation or stock deduction.
Suppliers / customers	Prototype	Session-scoped contact and status records; no foreign-key links to medicines, invoices or purchases.
Analytics	Prototype	Read-only sample metrics; no aggregation over operational transactions.

2.3.2 System context and data flow

The context diagram treats browser speech services as an external dependency. Their implementation and network use depend on the browser. The Django server receives the submitted text and selected language, not a microphone audio upload. The system does not automatically submit recognized speech; the user reviews the transcript and sends it explicitly.

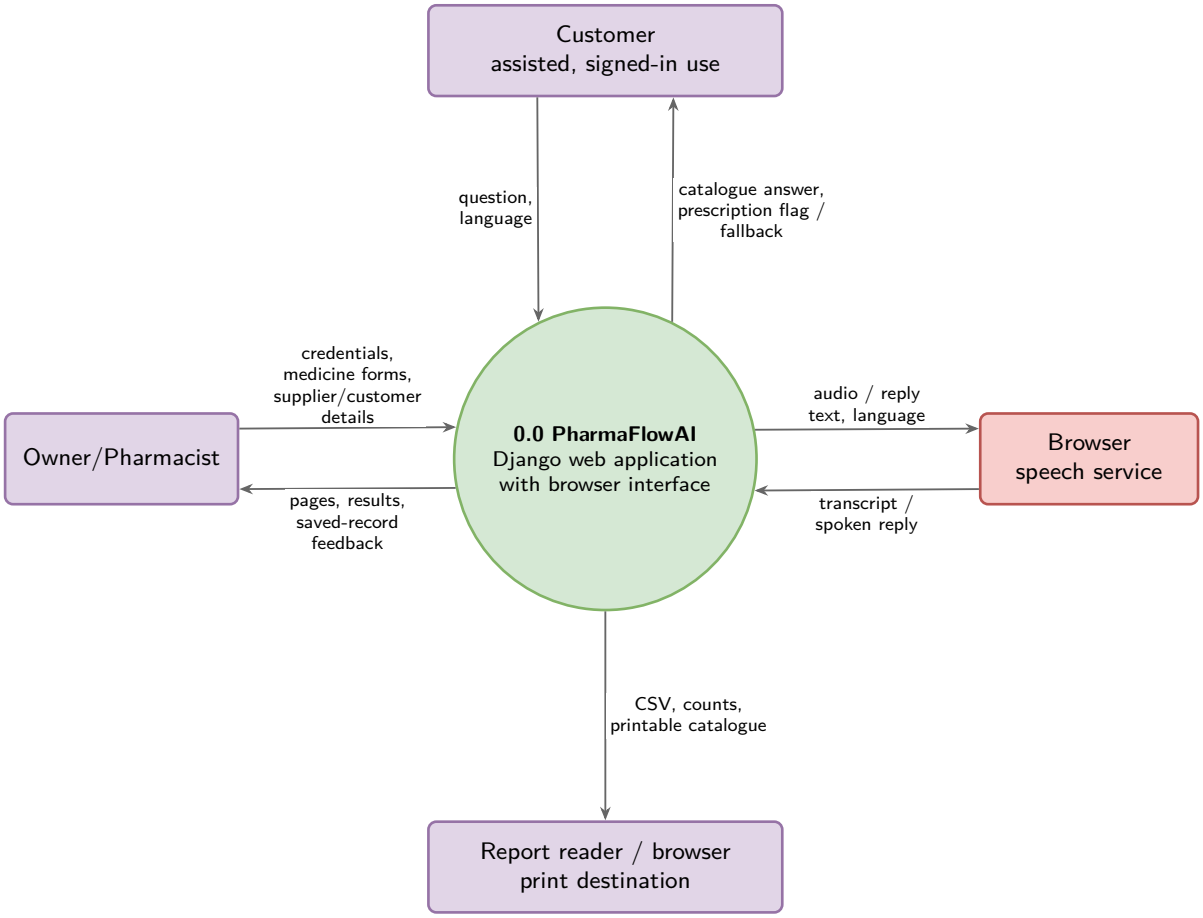


Figure 2.2: DFD Level 0. Customers use an authenticated counter session. Supplier details are entered by the Owner/Pharmacist; no supplier portal exists. Audio is processed through browser speech services, not uploaded to Django.

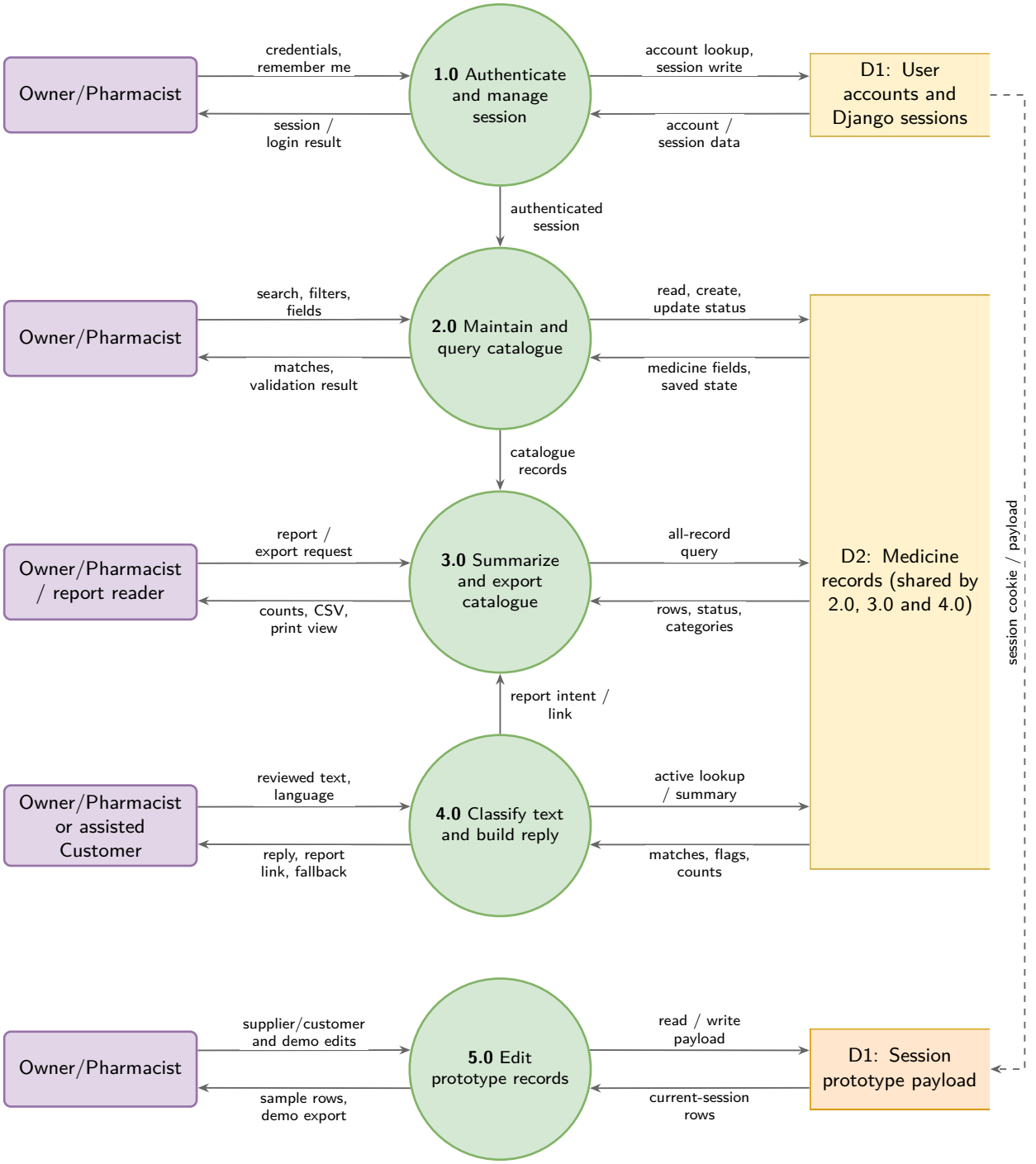


Figure 2.3: DFD Level 1. Repeated actors and D2 symbols denote the same participant or store. D1 contains framework sessions and the prototype payload, not an operational ledger. Processes 2.0–5.0 require an authenticated session; speech conversion surrounds process 4.0 in the browser.

2.3.3 Data model and data dictionary

Two project-defined domain models are migrated: `accounts.User` and `medicines.Medicine`. Django adds framework tables for sessions, permissions, groups, content types and administration logs. No batch, invoice, customer or supplier domain models exist. No direct user-to-medicine foreign key exists.

Table 2.4: Medicine entity data dictionary

Attribute	Type / size	Meaning and constraint
`id`	BigAutoField	Automatically generated primary key.
`name`	Char(200)	Required brand/product name; indexed.
`generic_name`	Char(200)	Optional composition or generic name.
`strength`	Char(50)	Optional pack-strength text; not a dosage recommendation.
`category`	Char(100)	Optional free-text classification; indexed.
`manufacturer`	Char(150)	Optional manufacturer label.
`unit`	Char(20), choice	Tablet, capsule, syrup, injection, ointment, drops, sachet or other; default tablet.
`prescription_required`	Boolean	Recorded prescription flag; default false. It is not legal or clinical verification.
`is_active`	Boolean	Default true; inactive records remain readable.
`created_at`, `updated_at`	Date/time	Creation and latest-save timestamps.

Medicine text is normalized by trimming and collapsing whitespace. The combination of lowercased name, strength and manufacturer is unique at database level. The form provides a corresponding duplicate validation message. A composite index supports active-status/name access. There are no quantity, stock balance, expiry, cost or selling-price fields in this entity.

The custom User model extends Django's `AbstractUser`. It retains username, password hash, identity fields and framework permission attributes; email is unique and required by the model; `role` is a 20-character choice field defaulting to OWNER. Django's standard groups and permissions remain available. This legacy role metadata does not define separate business actors in this SRS: Owner/Pharmacist is one functional actor. Stored labels are not treated as an enforced authorization policy.

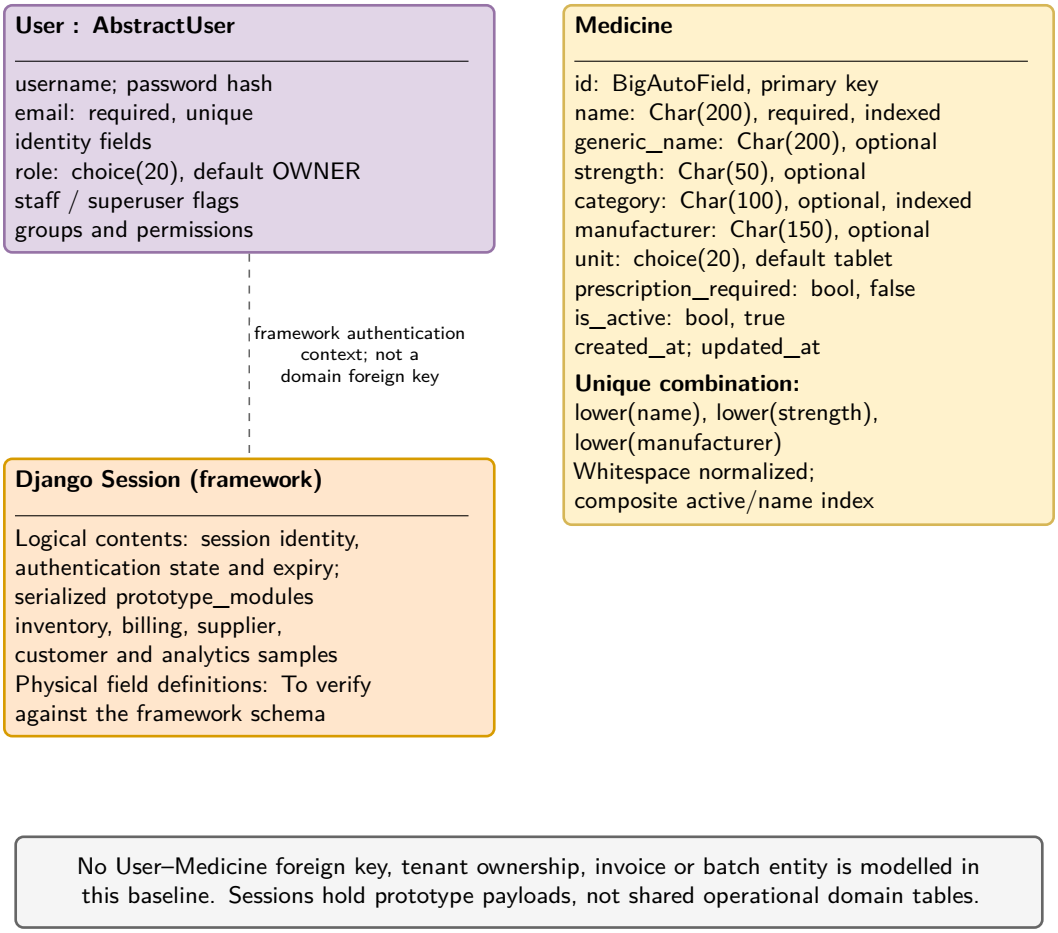


Figure 2.4: Class/ER view of the real domain and framework session store. Session attributes are shown conceptually because the source SRS does not enumerate its physical schema.

2.4 Operating Environment

Table 2.5: Verified technology baseline and conditional dependencies

Component	Baseline
Runtime / framework	Python 3.13.9 and Django 6.1 observed in the local environment; the development launcher selects Python 3.13.
Declared dependencies	Django 6.1, python-dotenv 1.1.1 and psycopg[binary] 3.2.10 in `requirements.txt`. Dependency installation on every platform is not verified.
Frontend	Django templates, HTML, CSS design tokens and plain JavaScript. Node.js runs assistant unit tests; it is not the application's web backend.
Database	SQLite by default; PostgreSQL backend selected when `POSTGRES_DB` is supplied. No production database deployment is assumed.
Web serving	Local Django development server; WSGI and ASGI entry points exist. Production server and hosting provider are unspecified.
Speech	Browser `SpeechRecognition`/`webkitSpeechRecognition` and `speechSynthesis`; availability and voices vary by device.
Client / hardware	A supported browser and display/keyboard for text use. Microphone and permission for voice input; speaker/headphones for audio output. No measured minimum CPU/RAM specification is established.
Network / configuration	Localhost for development or HTTPS for deployed microphone use. Browser speech services may require internet. Optional `.env` configuration; default time zone Asia/Kolkata.

Windows is the evidenced development environment. Other platforms may run the underlying technologies, but cross-platform acceptance testing is not evidenced. Speech input is an optional enhancement: unsupported browsers must retain text interaction. Security settings available outside debug mode include HTTPS redirection, secure cookies and HSTS; their correct deployment depends on configuration and a trusted reverse-proxy arrangement.

2.5 Design

2.5.1 Architecture and deployment boundary

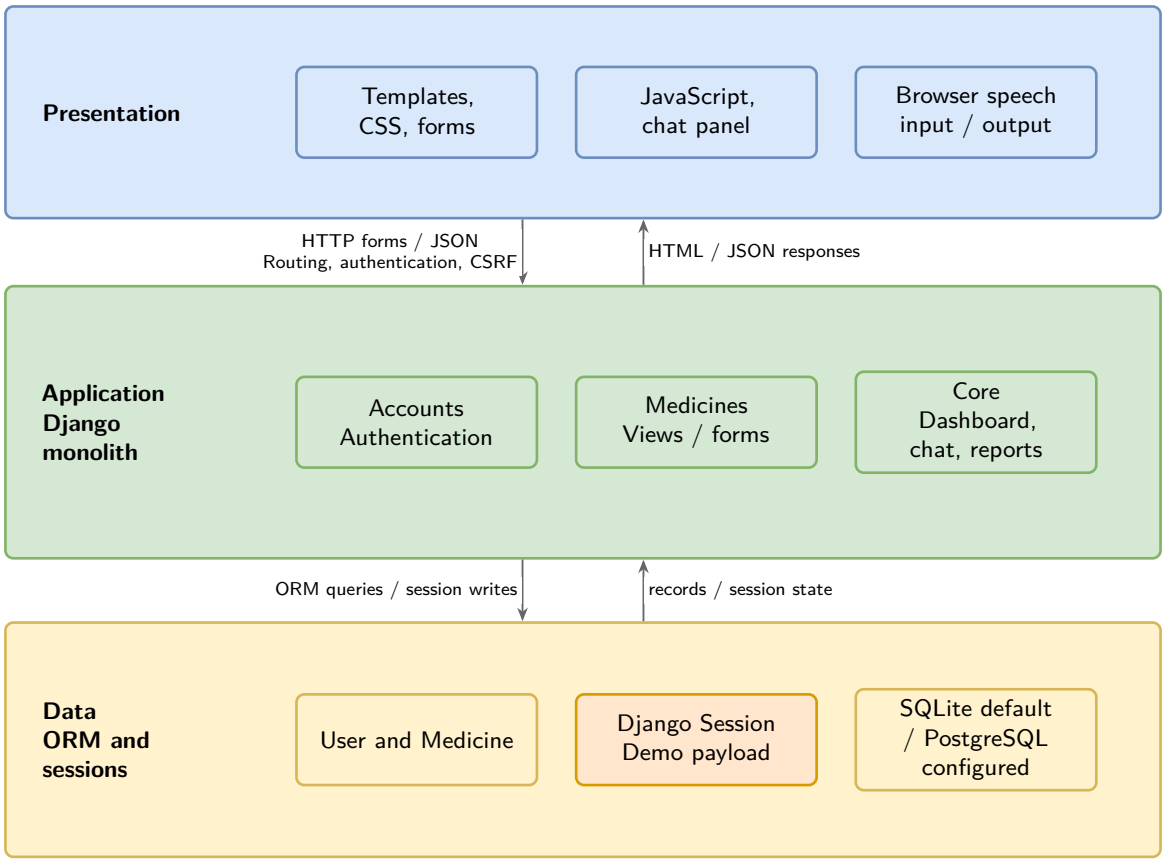


Figure 2.5: Layered component architecture. All application modules share the Django monolith. WSGI/ASGI hooks and PostgreSQL configuration do not establish production deployment.

2.5.2 Design and Implementation Constraints

- **Framework/runtime:** retain the recorded Django monolith and Python 3.13 development family; the observed versions are Django 6.1 and Python 3.13.9. No additional API framework or frontend application is assumed.
- **Persistence:** SQLite is the default; PostgreSQL is configurable but its production deployment is not established. Operational demos stay session-scoped until a separate domain-model change is accepted.
- **Assistant:** classification uses explicit rules, aliases and catalogue queries, with no external LLM. Browser speech availability, language voices, permission and possible online processing constrain voice use.
- **Development evidence:** Windows is the evidenced host; other platforms and minimum hardware specifications are To verify.
- **Academic timeline:** delivery must fit the course schedule, but deadlines and milestones are To verify and are not invented here.
- **Tools/cost:** free/open-source development tools are the requested planning constraint; a complete dependency/licence inventory and any hosting or speech-provider charges are To verify. No zero-cost deployment claim is made.

2.5.3 External interfaces

The primary user interface consists of sign-in, dashboard, medicine list/detail/form, prototype module, catalogue report and voice-assistant panel screens. Forms include CSRF tokens. JavaScript progressively enhances navigation, table search/export, password visibility and the assistant. The assistant requires JavaScript; ordinary server-rendered catalogue operations are not a single-page application.

Table 2.6: Principal HTTP interfaces

Route	Methods	Purpose / access
`/accounts/login/`	GET, POST	Display sign-in and authenticate.
`/accounts/logout/`	POST	End the session; GET is disallowed.
`/accounts/password-help/`	GET	Explain manual reset through administration.
`/dashboard/`	GET	Authenticated dashboard.
`/medicines/`	GET	Authenticated search/filter/sort list.
`/medicines/add/`, `/medicines/<id>/edit/`	GET, POST	Authenticated creation / editing.
`/medicines/<id>/`, `/medicines/<id>/status/`	GET; POST	Inspect record; change status respectively.
`/modules/<slug>/`	GET	Authenticated prototype view. Separate `save/` and `delete/` routes accept POST.
`/reports/medicine-catalogue/`	GET	Authenticated full report; `?format=csv` exports CSV.
`/chat/`	POST	Authenticated JSON assistant; CSRF required.
`/admin/`	Framework routes	Django administration; staff status and permissions apply.

The assistant request object contains a string `message` and optional `language`, which defaults to `en-IN`; `hi-IN` is the other allowed value. After trimming, messages must contain 1–500 characters. The view rejects request bodies larger than 8,192 bytes. Successful responses contain `reply`, `medicines` and, for report intents, `report_url`. Each medicine result contains name, generic name and prescription flag. The frontend accepts a report link only when it equals the server-rendered, same-application report path.

Validation errors use HTTP 400; the assistant view uses 401 when authentication is missing; CSRF rejection uses 403; oversized bodies checked by the view use 413. Django may reject requests earlier through its own request-size handling. Report and successful chat responses specify `Cache-Control: no-store`. Browser printing can save a PDF where supported; no server-side PDF generation service is implemented.

2.5.4 Catalogue activity

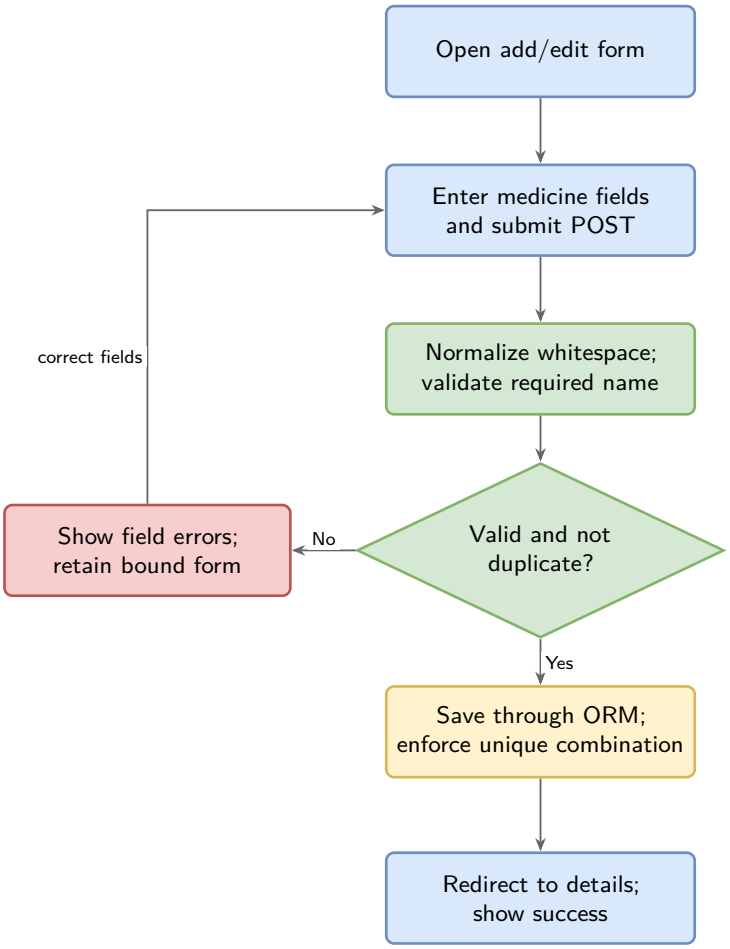


Figure 2.6: Catalogue creation/edit activity. Invalid submissions retain the form. Separate POST status actions deactivate or reactivate records; application forms do not delete medicines.

2.5.5 Voice-assistant interaction

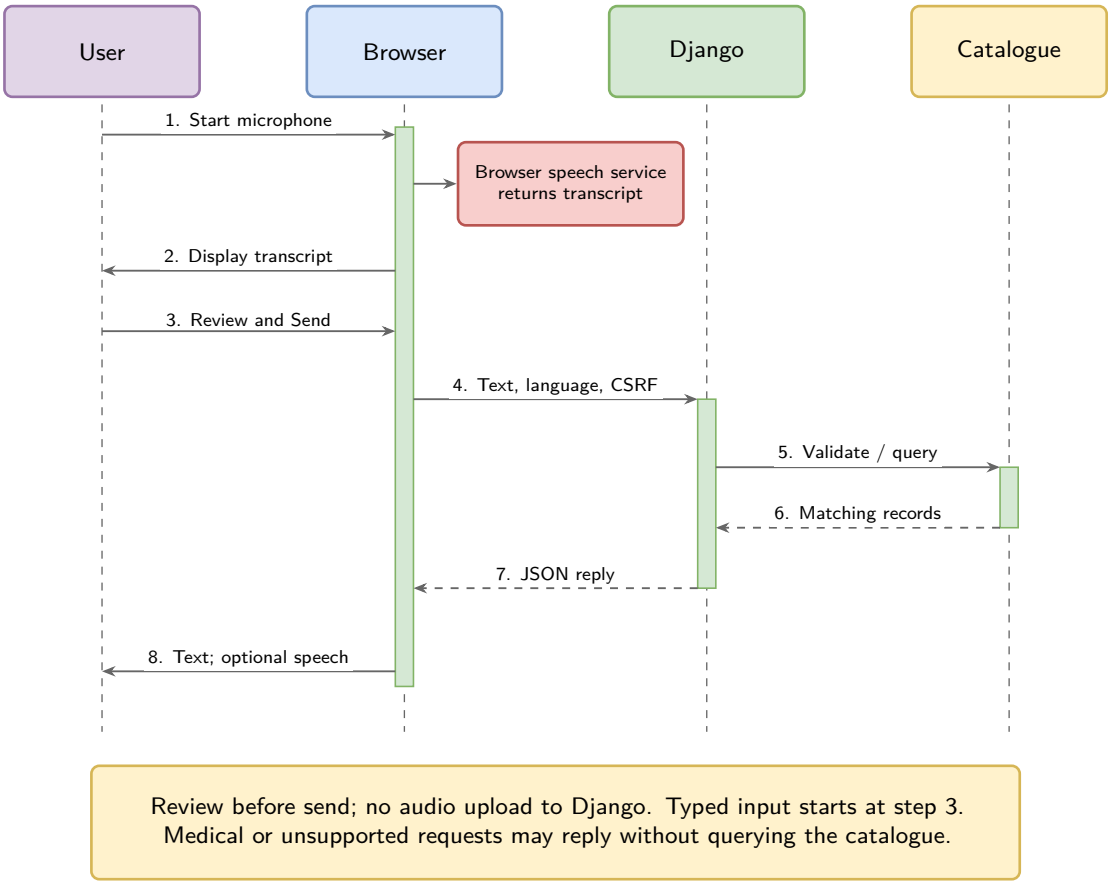


Figure 2.7: Voice-assistant sequence with activation bars. Recognition and speech synthesis are browser-dependent; Django receives reviewed text and language only.

The backend classifies text using explicit patterns, aliases and token matching. It does not retain conversation context or call a language model. A single lookup returns up to five active medicine results, while report requests summarize all active and inactive catalogue entries. Medical-pattern handling is a bounded rule set rather than a validated clinical classifier. Speech is single-turn, with review before transmission and optional playback afterward.

2.5.6 State behaviour

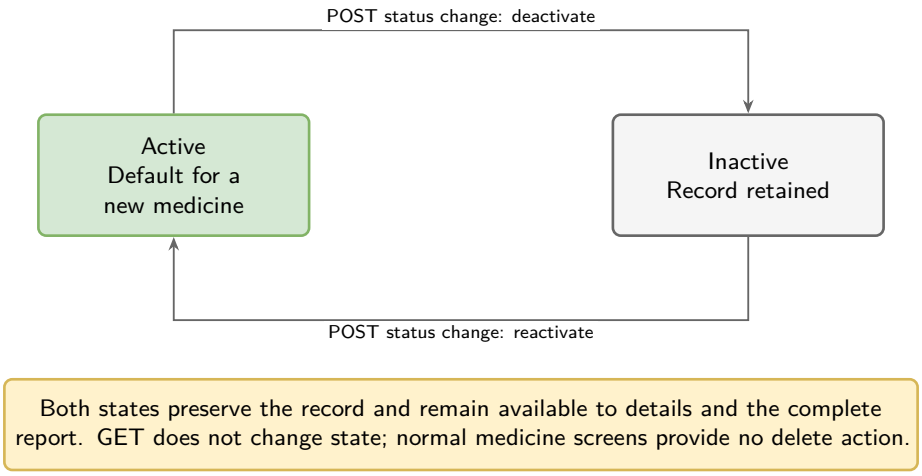


Figure 2.8: Medicine state diagram. POST status actions update the flag and timestamp without deleting the record.

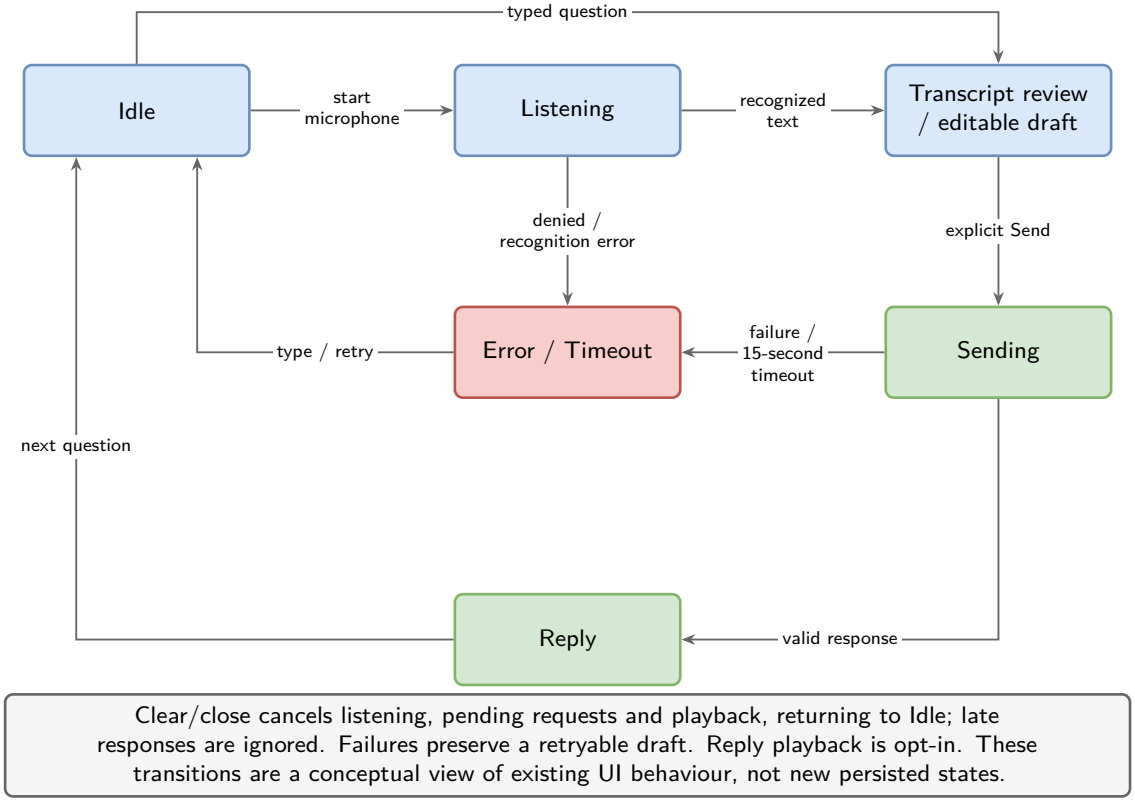


Figure 2.9: Assistant request states. Typed input bypasses Listening; recognition never sends automatically.

3 System Features

3.1 Description and Priority

Features are prioritized according to their role in a usable catalogue application. High priority denotes core access, data integrity and supported outputs; medium priority denotes convenience or demonstration functions. Prototype priority does not imply readiness for real transactions.

Table 3.1: Feature priorities

Feature	Priority	State	Rationale
Authentication	High	Implemented	Establish access to shared pharmacy data and end sessions correctly.
Medicine maintenance	High	Implemented	Maintain the master data used by catalogue views and reports.
Search and dashboard	High	Implemented	Locate records and inspect actual catalogue counts.
Catalogue report	High	Implemented	Provide a complete, exportable register with clear data limitations.
Voice/text assistant	Medium	Implemented	Offer bilingual catalogue access without promising live stock or medical advice.
Operational modules	Medium	Prototype	Demonstrate proposed workflows for later domain-model development.
Transactional inventory / billing	Medium	Proposed	Requires models, integrity rules and business decisions absent from the current system.

3.2 Functional Requirements

The acceptance criteria below define concrete checks against the reviewed baseline. They are not a statement that every manual scenario, environment or concurrency condition has already passed. The verification mapping in Section 5.3 distinguishes automated evidence from remaining validation.

3.2.1 Accounts and authentication

All requirements in Table 3.2 are implemented and high priority, except password-help guidance, which is medium priority. Sources: S02, S03 and S09 in Section 1.5.

Table 3.2: Authentication requirements and acceptance criteria

ID	Requirement	Priority	Status
FR-AUTH-01	The system shall authenticate an active account using a username or email and a valid password.	High	Implemented
Acceptance / verification: Valid credentials reach the dashboard or an accepted local next-page URL. An email lookup is case-insensitive.			
FR-AUTH-02	The system shall reject incorrect credentials and inactive accounts without authenticating them.	High	Implemented
Acceptance / verification: Incorrect or unknown credentials produce a generic failure message; an inactive user does not acquire an authenticated session.			

ID	Requirement	Priority	Status
FR-AUTH-03	The system shall apply the selected Remember me session policy.	High	Implemented
Acceptance / verification: Checked: session expiry uses the configured age, default 14 days. Unchecked: expiry is set to browser-close semantics. Browser restoration behaviour is outside the application guarantee.			
FR-AUTH-04	The system shall protect dashboard, medicine, module and report access through login requirements.	High	Implemented
Acceptance / verification: Anonymous requests redirect to login. Anonymous chat POST requests return HTTP 401 when not rejected earlier by CSRF middleware.			
FR-AUTH-05	The system shall accept sign-out through POST and flush the authenticated session.	High	Implemented
Acceptance / verification: GET logout is rejected; POST logout returns to sign-in with confirmation and subsequent protected access requires login.			
FR-AUTH-06	The system shall restrict post-login and post-status-change redirects to accepted local destinations.	High	Implemented
Acceptance / verification: A normal local next URL is honoured; an external next URL is ignored in favour of the default destination.			
FR-AUTH-07	The password-help page shall explain the manual administrator reset path.	Medium	Implemented
Acceptance / verification: The page states that self-service email recovery is unavailable; it does not claim to send a reset email.			

3.2.2 Medicine catalogue and dashboard

All requirements in Table 3.3 are implemented and high priority. Sources: S04, S05, S06 and S09.

Table 3.3: Catalogue and dashboard requirements

ID	Requirement	Priority	Status
FR-MED-01	The system shall create a medicine with its supported master-data fields and require a non-blank name.	High	Implemented
Acceptance / verification: A valid submission creates a database record and redirects to details. A missing or whitespace-only name displays validation errors.			
FR-MED-02	The system shall normalize medicine text and reject duplicate name/strength/manufacturer combinations case-insensitively.	High	Implemented
Acceptance / verification: Extra internal and edge whitespace is normalized; an equivalent duplicate fails validation. Different strength or manufacturer is allowed.			
FR-MED-03	The system shall permit inspection and editing of an existing medicine.	High	Implemented
Acceptance / verification: Details show stored data; the edit form is prefilled; a valid edit persists. A nonexistent identifier returns HTTP 404.			
FR-MED-04	The application shall toggle active/inactive status through POST without deleting the medicine.	High	Implemented
Acceptance / verification: POST changes the flag, updates the timestamp and shows confirmation. GET does not mutate state; reactivation is possible.			
FR-MED-05	The list shall search medicine name, generic name and manufacturer and filter by status and category.	High	Implemented
Acceptance / verification: A matching query narrows results; default status is active. Invalid form filters fall back to the defined defaults.			

ID	Requirement	Priority	Status
FR-MED-06	The list shall support the defined sort options and paginate at 20 records per page.	High	Implemented
Acceptance / verification: Name ascending/descending, category and recently updated ordering work; filters remain in pagination links.			
FR-MED-07	The interface shall distinguish an empty catalogue from a search with no matches.	High	Implemented
Acceptance / verification: An empty catalogue offers initial creation; a filtered empty result indicates that no match was found.			
FR-DASH-01	The dashboard shall derive total, active and inactive medicine counts from database records.	High	Implemented
Acceptance / verification: Counts agree with the current Medicine table; recent additions are the five latest records by creation time.			
FR-DASH-02	The dashboard shall identify operational modules as sample/demo views.	High	Implemented
Acceptance / verification: Module links and explanatory text do not represent sample rows or analytics values as completed transaction records.			

Catalogue dashboard categories count distinct non-empty recorded categories. The full report additionally groups blank categories under "Uncategorised". These different definitions are preserved rather than silently forcing their totals to agree.

3.2.3 Catalogue reporting

Table 3.4 requirements are implemented and high priority. Sources: S06 and S09.

Table 3.4: Catalogue reporting requirements

ID	Requirement	Priority	Status
FR-REP-01	The report shall include all active and inactive catalogue entries with a generation timestamp and summary counts.	High	Implemented
Acceptance / verification: Total, active, inactive and prescription-required counts agree with the loaded records; categories include an Uncategorised group when necessary.			
FR-REP-02	The report shall list name, composition, strength, category, manufacturer, unit, prescription flag and status.	High	Implemented
Acceptance / verification: Records are ordered by name, strength and identifier. Missing display details use a dash; the report explains that quantities, prices and expiry dates are absent.			
FR-REP-03	The report shall offer an authenticated UTF-8 CSV download with a header and every catalogue record.	High	Implemented
Acceptance / verification: Download is named medicine-catalogue.csv; CSV escaping preserves fields. Formula-like text is prefixed to avoid spreadsheet interpretation.			
FR-REP-04	The report shall provide a browser print action and a print stylesheet.	High	Implemented
Acceptance / verification: Print layout hides application navigation and assistant controls and requests landscape A4. Saving PDF depends on browser print support, not a server PDF API.			

3.2.4 Prototype operational functions

All requirements in Table 3.5 describe the current prototype and are medium priority. Source: S05. They must not be used as acceptance criteria for a completed pharmacy transaction system.

Table 3.5: Prototype module requirements

ID	Requirement	Priority	Status
FR-DEMO-01	The module view shall initialize its sample rows in the current session when absent.	Medium	Prototype
Acceptance / verification: Inventory, billing, suppliers, customers and analytics display their configured sample rows. One session's edits do not become shared domain records.			
FR-DEMO-02	Writable demo modules shall support add, edit and delete through POST.	Medium	Prototype
Acceptance / verification: Required non-empty fields are checked, valid mutations update the session payload and confirmation appears. Unknown edit identifiers report an error.			
FR-DEMO-03	Analytics shall remain read-only, and module tables shall support client-side search and export.	Medium	Prototype
Acceptance / verification: Analytics has no add/edit controls and mutation routes do not change it. Table search hides non-matching rows; export reflects visible table rows.			

Prototype validation primarily checks whether fields are non-empty. Numeric HTML inputs do not establish server-side quantity constraints. Status and amount fields are strings, not validated transaction states or monetary values. The generic demo-table export is separate from the catalogue CSV implementation and does not have the same explicit formula-neutralization logic. Production hardening of prototype exports remains proposed.

3.2.5 Voice and text assistant

Requirements in Table 3.6 are implemented. Access, validation and catalogue accuracy are high priority; language and speech convenience functions are medium priority. Sources: S07, S08, S09 and S10.

Table 3.6: Voice-assistant requirements

ID	Requirement	Priority	Status
FR-VA-01	The assistant shall accept authenticated text questions in English or Hindi with the defined request schema.	High	Implemented
Acceptance / verification: Unsupported language values, malformed JSON, non-string messages and empty or overlong text fail validation without a successful reply.			
FR-VA-02	Medicine questions shall use active catalogue records rather than fixed sample stock answers.	High	Implemented
Acceptance / verification: A known name or supported generic-name query returns at most five results. An inactive record is excluded from ordinary lookup; an unknown query receives a fallback.			
FR-VA-03	Recognized catalogue/report requests shall return current catalogue counts and the signed-in report link.	High	Implemented
Acceptance / verification: A request such as "medicine catalogue proper report" returns a summary and the correct local route rather than attempting only a medicine-name match.			
FR-VA-04	The assistant shall avoid claiming real stock, prices or treatment suitability from catalogue presence.	High	Implemented
Acceptance / verification: Catalogue responses state the stock/price limitation. Recognized medical-advice patterns request pharmacist assistance from the combined Owner/Pharmacist actor; no treatment-generating service is invoked. The combined actor label does not itself establish a clinical qualification.			
FR-VA-05	The interface shall offer English and Hindi selection and common Hindi medicine aliases.	Medium	Implemented
Acceptance / verification: Selecting the language changes assistant controls and reply language; an implemented alias maps to its English search term. Arbitrary transliteration is not promised.			

ID	Requirement	Priority	Status
FR-VA-06	Voice input shall start only following user action and shall place recognized words into an editable input.	Medium	Implemented
Acceptance / verification: A recognition event populates the draft without a chat request. Sending the reviewed draft triggers the request. Closing, hiding or leaving the page stops listening.			
FR-VA-07	Spoken replies shall be opt-in and use the selected language where browser voices support it.	Medium	Implemented
Acceptance / verification: Unchecked read-aloud produces no speech request. Checked read-aloud invokes synthesis; stop, uncheck, close and page lifecycle controls cancel playback.			
FR-VA-08	Text input shall remain available when voice is unsupported or permission is denied.	Medium	Implemented
Acceptance / verification: Unsupported recognition disables the microphone control with guidance. Denial and recognition failure release the listening state and allow typing.			
FR-VA-09	The assistant shall handle network failure, expired sessions and cancellation without inserting obsolete replies.	Medium	Implemented
Acceptance / verification: Requests are aborted after the configured 15-second client timeout. Failures preserve the draft; cleared/closed conversations ignore late responses.			
FR-VA-10	Chat history shall remain in page memory and be clearable.	Medium	Implemented
Acceptance / verification: Clear chat resets visible messages; navigation/reload does not restore them. The frontend retains at most 40 message elements; Django does not intentionally persist a conversation history.			

The privacy requirement covers application storage behaviour, not the retention policies of a browser speech provider. Voice processing may use an online browser service, and the interface discloses this. Pattern-based medical routing is not exhaustive understanding of all possible questions.

3.2.6 Use-case diagram

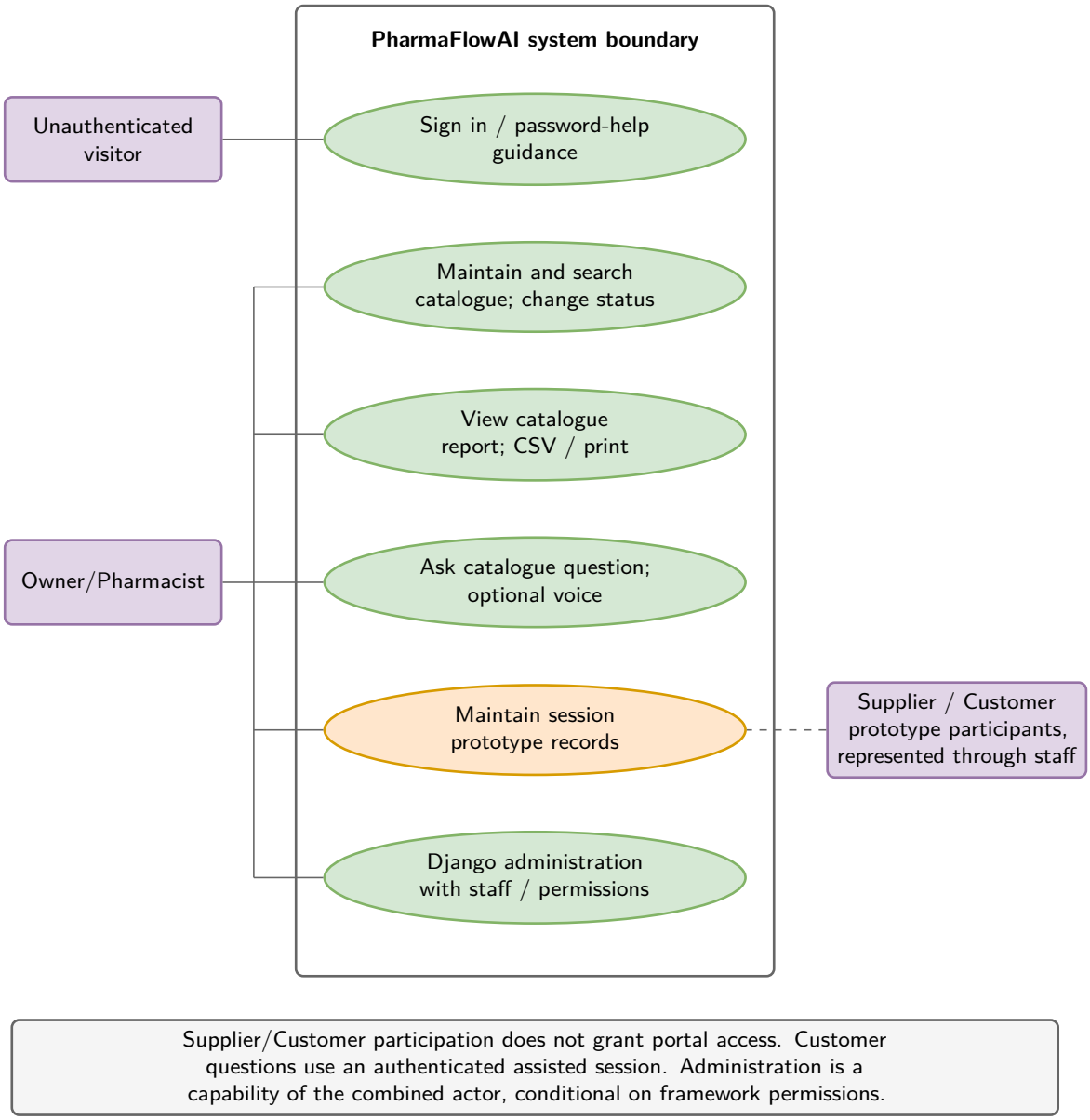


Figure 3.1: Use cases and access boundary. The dashed prototype association denotes business participation, not direct system authorization.

3.2.7 Use cases and detailed scenarios

Table 3.7: Representative use-case specifications

Use case	Scenario, alternatives and outcome
UC-01 Sign in	**Actor:** existing active user. **Precondition:** account has been created. **Trigger:** submit sign-in. **Flow:** enter email/username and password, select session preference, validate credentials, establish session, redirect. **Alternative:** invalid credentials show a generic error; unsafe next destinations are ignored. **Outcome:** authenticated session or no authenticated state. Maps to FR-AUTH-01–06.
UC-02 Maintain medicine	**Actor:** signed-in user. **Trigger:** Add medicine or Edit. **Flow:** enter master data, normalize and validate, check uniqueness, save, show details and confirmation. **Alternative:** missing/duplicate name prevents a successful save. **Outcome:** persisted valid record. Status changes are separate POST actions. Maps to FR-MED-01–04.
UC-03 Request report	**Actor:** signed-in user. **Trigger:** report link or assistant catalogue-report question. **Flow:** compute summary, open report, inspect complete register, optionally download CSV or print. **Alternative:** an empty catalogue shows a zero summary and empty-state message; unauthenticated report access redirects. **Outcome:** a timestamped report of current catalogue records, not stock or financial balances. Maps to FR-REP-01–04 and FR-VA-03.
UC-04 Ask by voice	**Actor:** user at a signed-in session. **Precondition:** supported browser, secure context and microphone permission. **Flow:** select language, start microphone, speak a name, review/correct transcript, send, receive catalogue reply, optionally hear it. **Alternatives:** denial/unsupported voice uses text; medical-pattern query requests pharmacist assistance from the Owner/Pharmacist; unknown medicine uses fallback. **Outcome:** bounded catalogue assistance; no order or prescription is created. Maps to FR-VA-01–10.
UC-05 Edit demo record	**Actor:** signed-in user. **Trigger:** open a writable module and choose Add/Edit. **Flow:** supply required fields, save and view changed sample row. **Alternative:** analytics is read-only; missing fields produce feedback; absent record IDs do not update a different record. **Outcome:** only the current session payload changes. Maps to FR-DEMO-01–03.

3.2.8 Acceptance Criteria for the Current Baseline

The following acceptance checklist defines completion of the current catalogue-and-assistance scope, not a transactional pharmacy system. Historical automated results are recorded in Section 5.3; remaining manual/environment checks stay To verify.

1. Valid username/email login creates the selected session policy; invalid/inactive credentials fail; protected pages and chat reject anonymous access.
2. Medicine creation, reading, editing and deactivation/reactivation persist correctly. Name validation and normalized duplicate constraints hold; application deletion is not part of catalogue CRUD here.
3. Search, status/category filters, supported ordering, 20-record pagination and dashboard counts agree with known fixture data.
4. The complete report includes active/inactive records, correct summaries and every CSV row with formula-neutralization. Target-browser print layout is manually checked.
5. English/Hindi catalogue lookup, report intent and medical-pattern fallback behave as specified; reviewed text is sent explicitly and optional voice retains text fallback.
6. Both recorded test commands complete with zero failures. The preserved baseline evidence is 85 Django and 9 JavaScript tests passed on 12 October 2026; this revision does not assert a new run.
7. Prototype modules remain labeled and session-scoped, analytics stays read-only, and no sample figure is

represented as a real transaction or inventory balance.

Validation acceptance target (To verify). Across an agreed fixture matrix, reject 100 percent of blank/whitespace-only names and normalized duplicate name/strength/manufacturer combinations; accept distinct strength/manufacturer combinations. Duplicate tests exist in the baseline, but exhaustive coverage and this explicit percentage target have not been measured.

3.2.9 Known Limitations

- No completed billing transactions, payment/tax processing, stock deduction or persistent inventory/batch records.
- No verified availability, price management, expiry calculation or clinical treatment decision support.
- No generative AI, persistent conversation history or unrestricted Hindi transliteration; medical routing is pattern-based and non-exhaustive.
- No supplier/customer portals, enforced permissions derived solely from the stored role label, or email password recovery.
- No demonstrated production PostgreSQL deployment, load capacity, backup/restore objective or cross-platform certification.
- Prototype exports lack the catalogue CSV's explicit formula-neutralization. Full reports load the catalogue into memory; speech services and browser print output require environment-specific verification.

3.2.10 Key Features Summary

Implemented: login, medicine catalogue/search, catalogue report/CSV/print and bounded bilingual assistance. Prototype: session operational records and sample analytics. Proposed/To verify: production hardening, persistent operations, scale and environment-specific acceptance.

4 Other Nonfunctional Requirements

4.1 Performance Requirements

No load-test report, throughput measurement or production latency objective is present in the project. The implemented list pagination, database indexes and bounded assistant input/result counts are design controls, not proof of a particular capacity. Full-report generation loads the complete catalogue into memory, so its scalability requires measurement.

Table 4.1: Performance and capacity requirements

ID	Requirement	Priority	Status
NFR-PER-01	The medicine list shall paginate at 20 records and use the configured search/sort fields. Verify page boundaries and filter retention in catalogue view tests.	Medium	Implemented
Acceptance / verification: With 21 matching records, page 1 shows 20 and page 2 shows one; search/status/category selections remain in page links.			
NFR-PER-02	The assistant shall bound message size, candidate processing and displayed results; ordinary matching examines at most 40 selected candidates and displays at most five. Verify boundary inputs and result cardinality.	Medium	Implemented
Acceptance / verification: Submit boundary inputs and a dataset with more than 40 candidates; no successful reply exceeds five displayed medicine results. Candidate-count instrumentation remains To verify.			
NFR-PER-03	For a separately agreed benchmark of 10 concurrent users and 10,000 catalogue records, target a 95th-percentile response time of at most 2 seconds for list, detail and text-chat requests on a documented reference host. Exclude external speech latency; record hardware, database and test data. This target is not a measured achievement.	Medium	To verify
Acceptance / verification: To verify: proposed k6 HTTP benchmark, 10 concurrent authenticated users, 10,000 seeded records, 2-minute warm-up and 10-minute run; measure endpoint-specific p95, target at most 2 seconds. Reference host CPU/RAM/OS, Python/Django versions, database/version and dataset seed must be recorded and agreed before execution. Exclude browser speech.			
NFR-PER-04	Benchmark full report and CSV generation at the agreed catalogue size, with an initial target of completion within 10 seconds and no missing records. If memory grows beyond deployment limits, introduce streaming or pagination before increasing capacity claims.	Medium	To verify
Acceptance / verification: To verify: proposed k6 benchmark on the same documented host with 10,000 records; run 30 sequential full-report requests and 30 CSV downloads after warm-up, target p95 at most 10 seconds and 100 percent record completeness. Record peak server memory; acceptable memory ceiling and concurrency remain to be agreed.			

4.2 Security Requirements

Table 4.2: Security requirements and limitations

ID	Requirement	Priority	Status
NFR-SEC-01	Password verification shall use Django's password hashing/authentication functions, not plaintext comparisons. Stored credentials shall not be displayed in normal application responses. Verify backend usage and generic login failures.	High	Implemented
Acceptance / verification: All invalid-credential test cases leave the client unauthenticated; normal responses contain neither supplied passwords nor stored password hashes.			
NFR-SEC-02	State-changing forms and chat POST requests shall use CSRF protection. Login-required views shall reject anonymous access. Verify missing-token and unauthenticated requests.	High	Implemented
Acceptance / verification: For every protected page in the route inventory, anonymous GET redirects to login. Anonymous chat POST with a valid CSRF context returns 401; authenticated unsafe requests without a valid token return 403. Target: 100 percent of enumerated cases rejected; complete route-matrix execution is To verify.			
NFR-SEC-03	Session cookies shall use configured HttpOnly and SameSite policies; unsafe next-page URLs shall not cause external redirects. Verify configuration and redirect tests.	High	Implemented
Acceptance / verification: Every tested external next URL is rejected; a valid local URL is accepted. Inspect HttpOnly/SameSite settings separately from production HTTPS deployment.			
NFR-SEC-04	Non-debug deployment shall use an explicit secret key, allowed hosts, HTTPS and correctly configured proxy trust. Secure-cookie/HSTS options are present, but a deployment audit is required; default debug settings are not a production configuration.	High	To verify
Acceptance / verification: Conditional on the target environment; Before release, document and pass every applicable deployment-checklist item for secret key, hosts, TLS, cookie flags, HSTS and proxy trust; deployment evidence remains To verify.			
NFR-SEC-05	Catalogue CSV shall neutralize formula-like text, and chat messages shall be inserted with text-safe DOM operations. Report links shall match the expected local path. Verify malicious cell prefixes and an unexpected external report link.	High	Implemented
Acceptance / verification: Exercise formula-triggering CSV cells, malicious chat text and an external report URL; target zero executable formulas, DOM script insertions or accepted external report links in the agreed fixtures. Full security fixture coverage is To verify.			
NFR-SEC-06	Define and enforce operation permissions for the combined Owner/Pharmacist actor, covering catalogue mutations and reports with positive and negative tests. Any future supplier/customer portal shall receive only its explicitly authorized access. A role field alone shall not be considered enforcement.	High	Proposed
Acceptance / verification: Proposed: create an operation-by-permission test matrix; require all allowed cases to succeed and all denied cases to fail before calling the policy enforced.			
NFR-SEC-07	Add login/chat abuse controls and a structured audit trail if production use is approved. Current login/status logging is not a complete, tamper-resistant business audit trail. Specify retention and access rules before implementation.	High	Proposed
Acceptance / verification: Proposed: agree rate limits, audit-event schema, retention and access rules, then test each threshold and logged event. Numeric thresholds are to be agreed.			

Customers using a staff-authenticated counter screen share that screen's existing access context. The current application is not a restricted public kiosk. A customer-only permission boundary would require additional design and verification.

4.3 Software Quality Attributes

Table 4.3: Quality attributes and verifiable expectations

ID	Requirement	Priority	Status
NFR-REL-01	Ordinary catalogue operations shall retain saved records across sessions and preserve inactive records. Verify create/edit/status persistence. Prototype rows shall remain explicitly session-scoped.	High	Implemented
Acceptance / verification: Create, edit and toggle a record, then read it from another authenticated session; values and identifier remain intact. Confirm that prototype edits do not become shared domain rows.			
NFR-REL-02	Assistant failures shall restore usable controls and preserve a retryable draft; late cancelled responses shall not reappear after clearing. Verify simulated denial, expiry, cancellation and unavailable-service paths.	High	Implemented
Acceptance / verification: Simulate denial, expiry and cancellation; input controls recover, retryable drafts remain, and zero late replies are appended after clear/close. Network-path coverage is To verify.			
NFR-USA-01	Forms shall provide labels and validation feedback, lists shall distinguish empty states, and the assistant shall expose status messages and keyboard-operable controls. Verify keyboard paths and accessible names; no formal accessibility conformance claim is made.	Medium	Implemented
Acceptance / verification: Manually traverse sign-in, catalogue form and assistant with the keyboard; each input has an accessible name and every validation error is perceivable. Full manual accessibility results are To verify.			
NFR-USA-02	Speech shall supplement text. Verify English/Hindi controls on the target browser and device, including unsupported input and unavailable voices. Installed voice quality and recognition accuracy are external dependencies.	Medium	To verify
Acceptance / verification: Conditional on the target environment; Test one supported and one unsupported recognition environment plus denied microphone access; English/Hindi text remains usable in all cases. Record browser/device versions and available voices; live results are To verify.			
NFR-MAI-01	Accounts, medicines and core functions shall remain separated into Django applications; styling and behaviours shall reside in reusable templates/static assets. Verify module responsibilities and execute relevant regression suites after changes.	Medium	Implemented
Acceptance / verification: Inspect responsibilities of accounts, medicines and core, then run both recorded regression commands; acceptance requires zero failing tests. The historical counts remain 85 Django tests and 9 JavaScript tests.			
NFR-SCA-01	Validate concurrent catalogue updates and PostgreSQL migration before claiming multi-user production scale. Multi-store tenancy, distributed workers and horizontal scaling are outside the established baseline.	Medium	Proposed
Acceptance / verification: Proposed: agree concurrent-update workloads and a PostgreSQL migration rehearsal; record conflicts, errors and integrity results before asserting capacity.			
NFR-REC-01	Define and exercise a backup/restore procedure for users, catalogue and configuration before production adoption. Recovery time and permissible data loss must be agreed; neither is established by this repository.	High	Proposed
Acceptance / verification: Proposed, to be agreed: RPO (maximum permissible data loss) and RTO (maximum restoration time). In a restore drill, measure the recovered backup timestamp, elapsed restore time and row-count/integrity checks against the agreed RPO/RTO; no value or successful drill is established.			

ID	Requirement	Priority	Status
NFR-PRI-01	The application shall not deliberately persist voice audio or chat history, and it shall disclose possible online browser speech processing. Verify page-memory behaviour; provider retention and browser policies require separate review.	High	Implemented

Acceptance / verification: Application behaviour is implemented; external-provider behaviour remains conditional. Clear and reload the page and inspect application storage/network behaviour; no application conversation history is restored and Django receives text, not audio. Browser-provider retention remains To verify.

4.4 Business Rules

BR-1. A medicine is catalogue master data, not a batch or a stock balance. A record's existence must not be interpreted as availability for sale.

BR-2. Brand/name, strength and manufacturer together determine case-insensitive catalogue uniqueness after whitespace normalization.

BR-3. Catalogue records default to active; the ordinary list defaults to active records. Inactive records remain accessible through details, status filters and the complete report.

BR-4. The application medicine screens provide deactivation/reactivation rather than deletion. Django administration is a separate privileged interface and is not claimed to prohibit all deletion.

BR-5. Prescription flags reflect data entered into the catalogue. A false flag is reported cautiously and does not certify that a prescription is legally unnecessary.

BR-6. The voice assistant shall not create orders, prescribe medicines, determine treatment suitability or assert unrecorded prices and quantities.

BR-7. Operational sample records shall not be treated as accounting, inventory or customer-history evidence. A session expiration or logout may discard the session context.

BR-8. Only signed-in users can obtain application catalogue reports or use the assistant. Customer-facing wording does not create an anonymous access route.

BR-9. Report totals refer to catalogue records, including inactive records. Dashboard and report category definitions differ where blank categories exist.

BR-10. Data displayed in analytics is demonstrative. No revenue or profit claim may be derived from those predefined values as if calculated from transactions.

5 Other Requirements

5.1 Assumptions and Dependencies

- The baseline represents one shared pharmacy catalogue. Multiple tenant isolation is neither modelled nor verified.
- Accounts are provisioned through authorized administration; public registration is not implemented. Django administration privileges for the combined Owner/Pharmacist actor must be configured independently of the stored role label.
- Users enter accurate catalogue and prescription-flag data. The system does not validate it against a national drug database or regulatory service.
- Speech requires a compatible browser, user permission and a suitable environment. Network availability may be required by the browser's provider; text assistance must remain usable without voice.
- SQLite is the local default. Production database selection, hosting, monitoring and backup arrangements remain decisions to be documented.
- The supplied reference determines section organization and visual style, not this project's features. Neither reference supplies project features or implementation technologies for PharmaFlowAI.
- Author names and roll numbers, department, institution and course details were supplied for this revision. Supervisor and submission date remain editable placeholders as requested for this revision; the imported supervisor value is retained only in a source comment.

5.2 Maintenance and Proposed Enhancements

Maintenance shall keep implementation status visible as the prototype evolves. Database changes require Django migrations and appropriate regression tests. Dependency changes should be tested against authentication, catalogue constraints, reporting and browser behaviour. Secrets and user credentials must not be embedded in documentation or source-controlled configuration.

Table 5.1: Future work, explicitly outside the implemented baseline

ID	Requirement	Priority	Status
FUT-01	Persistent inventory and batches	Medium	Proposed
Acceptance / verification: Required design before acceptance: Batch identifiers, quantities, expiry and prices; validated stock movements and concurrency rules.			
FUT-02	Transactional billing	Medium	Proposed
Acceptance / verification: Required design before acceptance: Invoice lines, monetary calculation, payments, stock effects, returns and applicable business rules.			
FUT-03	Supplier/customer domain records	Medium	Proposed
Acceptance / verification: Required design before acceptance: Shared models, relationships, validation, access control and agreed retention.			
FUT-04	Derived analytics	Medium	Proposed
Acceptance / verification: Required design before acceptance: Defined metrics computed from validated transaction data rather than sample strings.			
FUT-05	Access control / recovery	Medium	Proposed
Acceptance / verification: Required design before acceptance: Explicit operation permissions and verified email-token password recovery.			
FUT-06	Expanded assistant	Medium	Proposed
Acceptance / verification: Required design before acceptance: Any generative model, wider language recognition or live-stock tool must have separately specified data access, cost, privacy and accuracy requirements. None is assumed here.			

5.2.1 Delivery Phases (Proposed planning sequence)

No delivery dates or additional completion claims are established. Each phase below is a Proposed plan; phase 1 reviews the already implemented baseline.

1. **Proposed** baseline acceptance: validate the implemented catalogue/report/assistant and close outstanding device and permission checks.
2. **Proposed** persistent inventory: design shared supplier/customer records, batches, quantities, prices and expiry/stock rules (FUT-01, FUT-03).
3. **Proposed** billing: add validated invoices, payment and stock effects only after inventory integrity is accepted (FUT-02).
4. **Proposed** analytics: derive agreed measures from validated transactions rather than sample strings (FUT-04).
5. **Proposed** expanded assistant: specify any additional language, live-stock or model integration, with separate data-access, privacy, accuracy and cost requirements (FUT-06).

5.3 Verification and Acceptance Mapping

Verification uses Django tests for server behaviour and Node.js tests that simulate frontend speech events. Simulation avoids collecting microphone audio; it does not establish live recognition quality. The following commands are the repeatable baseline checks:

```
py -3.13 manage.py test apps.core apps.accounts apps.medicines -noinput
node tests/assistant.test.cjs
```

Table 5.2: Requirement-to-verification mapping

Requirement group	Evidence in repository	Additional acceptance work
FR-AUTH; NFR-SEC-01–03	Account login tests: credentials, expiry, redirects, logout and anonymous access.	Validate browser session-restoration expectations and production HTTPS configuration.
FR-MED; FR-DASH	Medicine model/view tests: normalization, uniqueness, filters, pagination, state changes and counts.	Review real-world data entry, keyboard navigation and concurrent updates.
FR-REP	Core report tests: counts, full rows, CSV escaping, authentication and empty catalogue.	Visually check pagination and output on target browser print/PDF destinations.
FR-DEMO	Core module tests: sample-page rendering and customer add/edit/delete.	Verify remaining demo forms, visible export and intentional session isolation.
FR-VA-01–05	Core assistant tests: live catalogue queries, Hindi alias, medical-pattern routing, report intents and request validation.	Explore ambiguous queries and unsupported aliases; do not infer conversational intelligence.
FR-VA-06–10	Nine JavaScript tests: transcript review, fallback, denied access, close/clear cancellation, read-aloud, expiry and report-link checks.	Test microphone capture and audible English/Hindi output on the actual customer device.
Proposed NFRs	No benchmark/deployment proof established.	Execute the agreed load, permissions, recovery and platform validation plans before claiming compliance.

5.3.1 Detailed requirement traceability

Each TC identifier below is an acceptance scenario, not a newly executed test or a one-to-one claim about a source test function. "Passed in baseline" is used only for scenarios supported by the original document's stated automated evidence. A requirement may be implemented while an additional acceptance scenario is still To verify. Django evidence refers to S09; simulated Node evidence refers to S10. Manual evidence means a planned check, not a completed observation.

Table 5.3: Traceability: authentication, catalogue, reports and prototypes

Test ID	Scenario	Expected result	Requirement IDs	Evidence	Status
TC-01	Login using supported credentials	Valid username/email establishes an authenticated session.	FR-AUTH-01	Django test	Passed in baseline
TC-02	Invalid credentials	Generic error; no authenticated session.	FR-AUTH-02; NFR-SEC-01	Django test	Passed in baseline
TC-03	Inactive account	Authentication denied.	FR-AUTH-02	Django test (planned detail)	To verify
TC-04	Remember me policies	Configured age versus browser-close session semantics.	FR-AUTH-03	Django test	Passed in baseline
TC-05	Local / external next URL	Local destination accepted; external redirect rejected.	FR-AUTH-06; NFR-SEC-03	Django test	Passed in baseline
TC-06	Logout and protected access	Logout ends session; later protected requests require login.	FR-AUTH-04–05	Django test	Passed in baseline
TC-07	Create, read, edit medicine	Valid values persist and details reflect the saved record.	FR-MED-01,03; NFR-REL-01	Django test (planned detail)	To verify
TC-08	Required medicine name	Blank and whitespace-only names rejected in every agreed fixture.	FR-MED-01	Django test (planned detail)	To verify
TC-09	Normalization and duplicates	Whitespace normalized; equivalent name/strength/manufacturer rejected.	FR-MED-02	Django test	Passed in baseline
TC-10	Medicine status changes	Status toggles; record retained.	FR-MED-04	Django test	Passed in baseline
TC-11	GET status / missing record	GET does not mutate; nonexistent detail/edit ID returns 404.	FR-MED-03–04	Django test (planned detail)	To verify
TC-12	Search and filters	Matching fields and status/category select expected rows.	FR-MED-05	Django test	Passed in baseline
TC-13	Pagination	20-record page boundary and filter retention.	FR-MED-06; NFR-PER-01	Django test	Passed in baseline
TC-14	Empty and no-match list	Initial creation prompt differs from a filtered no-match message.	FR-MED-07	Manual	To verify
TC-15	Dashboard counts	Counts agree with Medicine records; sample values remain labeled.	FR-DASH-01–02	Django test (counts); Manual (labels)	To verify
TC-16	Full / empty catalogue report	All rows and expected counts; empty dataset handled.	FR-REP-01–02	Django test	Passed in baseline
TC-17	Authenticated CSV and escaping	Full CSV rows, proper field escaping; anonymous report access denied.	FR-REP-03; FR-AUTH-04	Django test	Passed in baseline
TC-18	CSV formula prefixes	Formula-like text is neutralized for every agreed malicious fixture.	FR-REP-03; NFR-SEC-05	Django test (planned detail)	To verify
TC-19	Print / save PDF	Navigation hidden; readable landscape A4 output on target browser.	FR-REP-04	Manual	To verify
TC-20	Prototype rendering and customer mutations	Sample pages render; customer add/edit/delete changes session payload.	FR-DEMO-01–02	Django test	Passed in baseline
TC-21	Other demos and session isolation	Remaining forms/export work; another session does not share mutations.	FR-DEMO-01–03	Manual	To verify
TC-22	Read-only analytics	No add/edit controls or successful mutation; figures remain samples.	FR-DEMO-03	Manual	To verify

Table 5.4: Traceability: chatbot, voice and outstanding nonfunctional checks

Test ID	Scenario	Expected result	Requirement IDs	Evidence	Status
TC-23	English catalogue lookup	Current catalogue data used for a recognized medicine query.	FR-VA-01–02	Django test	Passed in baseline
TC-24	Supported Hindi alias	Supported Hindi query maps to the expected catalogue search.	FR-VA-05	Django test	Passed in baseline
TC-25	Medical-pattern question	Bounded routing to pharmacist assistance; no treatment generated.	FR-VA-04	Django test	Passed in baseline
TC-26	Catalogue report intent	Counts and local report link returned.	FR-VA-03	Django test	Passed in baseline
TC-27	Request validation	Invalid request schema rejected rather than producing a successful reply.	FR-VA-01	Django test	Passed in baseline
TC-28	Exact validation boundaries	Empty/501-character text, invalid language, malformed JSON and non-string message return 400.	FR-VA-01; NFR-PER-02	Django test (planned detail)	To verify
TC-29	Anonymous chat with CSRF context	401 when not rejected earlier by CSRF.	FR-AUTH-04; NFR-SEC-02	Django test (planned detail)	To verify
TC-30	Missing CSRF token	Authenticated unsafe chat request rejected with 403.	NFR-SEC-02	Django test (planned detail)	To verify
TC-31	Oversized chat body	Body above 8,192 bytes rejected with 413 when handled by the view.	FR-VA-01; NFR-PER-02	Django test (planned detail)	To verify
TC-32	Transcript review	Recognition fills editable draft; no automatic chat request.	FR-VA-06	Node test	Passed in baseline
TC-33	Unsupported / denied recognition	Listening releases and typing remains available.	FR-VA-08	Node test	Passed in baseline
TC-34	Opt-in read-aloud	Playback follows opt-in controls and is cancellable.	FR-VA-07	Node test	Passed in baseline
TC-35	Expired session	Failure handled with usable controls.	FR-VA-09; NFR-REL-02	Node test	Passed in baseline
TC-36	Clear / close during request	Cancelled/late responses do not repopulate cleared chat.	FR-VA-09–10	Node test	Passed in baseline
TC-37	Client timeout	Pending request aborted after configured 15 seconds; draft remains retryable.	FR-VA-09; NFR-REL-02	Node test (planned detail)	To verify
TC-38	Untrusted report link	Unexpected external link is not accepted.	FR-VA-03; NFR-SEC-05	Node test	Passed in baseline
TC-39	Live English/Hindi speech	Actual device captures speech and plays selected-language output when supported.	FR-VA-05–08; NFR-USA-02	Manual	To verify
TC-40	Page-memory history cap	No restored chat after reload; at most 40 message elements retained.	FR-VA-10; NFR-PRI-01	Manual	To verify
TC-41	Performance and report completeness	Proposed p95 targets and record completeness met under the specified benchmark.	NFR-PER-03–04	Manual (k6 plan)	To verify
TC-42	Deployment, permissions and recovery	Agreed deployment checklist, operation matrix and backup/restore drill pass.	NFR-SEC-04,06–07; NFR-REC-01	Manual	To verify

Baseline verification note. Both suites were re-executed on 12 October 2026: all 85 Django tests and all nine JavaScript tests passed. Django's system check reported no issues. The launcher also emitted a Python-location warning, but the test run completed successfully. Automated test results are bounded evidence and do not establish production readiness.

5.4 Reference Alignment and Document Control

The original five numbered chapters and their existing subsections are retained. Definitions and references are consolidated into Sections 1.4 and 1.5, with pointers preserved in Sections 5.5 and 5.6. New third-level subsections add permissions, constraints, diagrams and acceptance planning without removing existing requirements.

The original S01 reference supplied the chapter hierarchy and academic presentation. The additional R04 reference supplies only visual and structural ideas: pastel diagram palettes, layered architecture, explicit diagram

boundaries and document-control information. All actors, data and requirements remain PharmaFlowAI-specific. DFDs represent logical information flows; the class/ER view represents the actual data model. Use-case and state diagrams clarify existing interactions rather than establish new features.

Serif body text, sans-serif numbered headings, A4 pages, contents, lists of figures/tables, numbered captions and page numbers are retained. Page count follows the content rather than a reference document's length. Version remains 1.0 and the implementation baseline remains 12 October 2026. This editorial revision does not claim new test results.

5.5 Source References

The complete source list S01–S10 is retained in Section 1.5. S01 records the original structural reference; R04 records the additional visual reference used for this revision.

5.6 Terminology

The original terminology is merged into the two-column definitions table in Section 1.4.